

ANALISIS MATEMATIS SISTEM KENDALI

MISO DUAL-LOOP PID PADA EXHAUST FAN DENGAN OVERRIDE LOGIC

Penyusun: Farrel Emmanuel Joseph Renyaan

NIM: 101022400025

Program Studi: Teknik Elektro (Industrial Automation)

Institusi: Telkom University

BAB 1: Pemodelan Matematis *Plant* (Motor DC / Kipas *Exhaust*)

Sistem fisik aktuator yang digunakan adalah Motor DC. Untuk memodelkan pergerakan motor di dalam simulasi (seperti MATLAB Simulink), kita menggunakan pendekatan persamaan diferensial dari sisi elektrik dan mekanik.

1. Persamaan Elektrik (Rangkaian Jangkar) Tegangan input yang diberikan ke motor menghasilkan arus yang melawan Gaya Gerak Listrik (GGL) balik. Persamaan Hukum Kirchhoff untuk tegangan dinamisnya adalah:

$$v(t) = R \cdot i(t) + L \frac{di(t)}{dt} + e_a(t)$$

Keterangan:

- $v(t)$ = Tegangan masukan PWM (Volt)
- R = Hambatan terminal motor (Ohm)
- L = Induktansi kumparan (Henry)
- $i(t)$ = Arus dinamo (Ampere)
- $e_a(t)$ = Tegangan GGL balik (Volt), di mana $e_a(t) = K_e \cdot \omega(t)$

2. Persamaan Mekanik (Rotasi Kipas) Torsi yang dihasilkan motor digunakan untuk melawan momen inersia baling-baling kipas dan gesekan udara (viskos). Persamaan Dinamika Newton untuk rotasinya adalah:

$$\tau(t) = J \frac{d\omega(t)}{dt} + B \cdot \omega(t)$$

Keterangan:

- $\tau(t)$ = Torsi motor (Nm), di mana $\tau(t) = K_t \cdot i(t)$
- J = Momen Inersia baling-baling kipas ($kg \cdot m^2$)
- B = Koefisien friksi viskos (Nms/rad)
- $\omega(t)$ = Kecepatan sudut baling-baling (rad/s)

3. Fungsi Alih (*Transfer Function*) Dengan melakukan Transformasi Laplace pada kedua persamaan diferensial di atas dan mengasumsikan kondisi awal nol, kita mendapatkan model matematika orde-2 yang merepresentasikan respons keterlambatan inersia kipas. Bentuk umum Fungsi Alih $G(s)$ antara tegangan input $V(s)$ terhadap kecepatan output $w(s)$ adalah:

$$G(s) = \frac{\omega(s)}{V(s)} = \frac{K_t}{(L \cdot s + R)(J \cdot s + B) + K_t \cdot K_e}$$

Untuk keperluan validasi simulasi *step-response* pada MATLAB Simulink, konstanta motor DC ini disederhanakan menjadi fungsi alih sistem orde-2 teredam kritis:

$$G(s) = \frac{1}{s^2 + 2s + 1}$$

BAB 2: Hukum Kendali PID Diskrit (*The Controller*)

Karena algoritma kendali ditanamkan ke dalam mikrokontroler (Arduino) yang bekerja berdasarkan *sampling rate* digital interval = 1000 milidetik, maka fungsi kendali PID yang awalnya berupa persamaan kontinu (integral/derivatif murni) ditransformasikan menjadi bentuk diskrit (Euler).

1. Definisi *Error* (Aksi *Reverse*) Sistem *exhaust* bersifat mendinginkan/membuang, sehingga menggunakan mode aksi *Reverse* (ketika *Process Variable* naik, *Output* aktuator harus naik untuk menurunkannya kembali).

$$e[k] = PV[k] - SP$$

2. Persamaan PID Diskrit Output dari masing-masing *loop* pengontrol (*Loop Asap* dan *Loop Suhu*) dihitung menggunakan deret matematika berikut di setiap siklus :

$$u[k] = K_p \cdot e[k] + K_i \cdot T_s \sum_{i=0}^k e[i] + \frac{K_d}{T_s} (e[k] - e[k-1])$$

Keterangan:

- $u[k]$ = Sinyal kendali output pada waktu ke- k (Nilai PWM)
- $e[k]$ = Nilai *Error* saat ini
- $e[k-1]$ = Nilai *Error* pada pembacaan sebelumnya
- T_s = Waktu *sampling* periodik (*Sampling Time*) sebesar 1 detik
- K_p, K_i, K_d = Konstanta *Tuning* PID

Dalam sistem ini, parameter *tuning* ditetapkan secara empiris menggunakan metode osilasi teredam dengan nilai:

- **Proportional (Kp) = 0.5:** Memberikan daya dorong awal (*kick*) yang halus tanpa menyebabkan lonjakan arus berlebih pada pin *Gate* MOSFET.
- **Integral (Ki) = 0.1:** Menghilangkan *Steady-State Error* secara gradual untuk memastikan ruangan benar-benar mencapai *Setpoint* tanpa *Integral Windup* berkat fitur saturasi limit (0 - 255).
- **Derivative (Kd) = 0.1:** Meredam laju perubahan kecepatan kipas agar aktuator motor DC tidak aus akibat sentakan perubahan voltase instan.

BAB 3: Arsitektur *Override Control* (MISO)

Keunikan arsitektur sistem MISO (*Multi-Input Single-Output*) ini terletak pada kehadiran fitur *Safety Override*. Sinyal kendali akhir yang disalurkan ke motor DC melalui fungsi `analogWrite()` bukanlah penjumlahan matematis, melainkan hasil komputasi *High-Selector Logic* (*Max Selector*).

Persamaan fungsi logikanya adalah:

$$U_{final} = \max(u_{asap}, u_{suhu})$$

Jika fungsi di atas diganti dengan penjumlahan linier $U_{final} = U_{asap} + U_{suhu}$, sistem berisiko mengalami fenomena saturasi dinamis lebih cepat (melewati batas bit PWM 255), yang akan menghancurkan linearitas kurva PID. Dengan fungsi $\max$, sistem mengevaluasi dan memilih variabel *Error* mana yang memiliki urgensi tingkat bahaya termal/polutan tertinggi pada siklus K tersebut, lalu mengalokasikan tenaga kipas maksimal hanya pada *loop* yang kritis.